

Jihoon Choi

✉ cjihoon1234@gmail.com | ☎ (672) 866-4158 | [in linkedin.com/in/jihoon-choi-60c5/](https://www.linkedin.com/in/jihoon-choi-60c5/) | github.com/JihoonChoi1

EDUCATION

University of British Columbia <i>Bachelor of Science in Computer Science; CGPA: 4.30/4.33</i> - Degree timeline includes mandatory national service (Aug. 2024 – May 2026). Dean's Scholar (Winter Session 2023/2024).	Expected May. 2028 <i>Vancouver, BC</i>
Langara College <i>Transfer Program to UBC; CGPA: 4.29/4.33</i> Dean's Honour Roll (April 2023).	May 2022 – Apr. 2023 <i>Vancouver, BC</i>

EXPERIENCE

Systems Administrator <i>Ministry of Justice, Republic of Korea — Suwon Immigration Office</i> - Supported immigration office operations under mandatory alternative military service, assisting daily workflows. - Improved workstation uptime by configuring BIOS RTC scheduling to automate boot sequences and reduce failures. - Hardened endpoint security on public systems via Registry policies, SRP, and ACLs to prevent unauthorized execution. - Automated user session cleanup by scheduling PowerShell scripts to delete temporary files and protect user privacy. - Developed a lightweight web-based QR code tool using open-source libraries to simplify photo distribution. - Assisted in naturalization and citizenship processes, including document digitization, verification, and visitor support. - Received Commissioner's Commendation from Gyeongin Regional Military Manpower Administration (Apr. 2026).	Aug. 2024 – May 2026 <i>Suwon, South Korea</i>
---	---

TECHNICAL PROJECTS

Bare-Metal Type-1 Hypervisor 🌀 <i>Rust (no_std), AArch64 Assembly, QEMU</i> - Built a bare-metal Type-1 hypervisor for ARMv8-A (EL2) from scratch using no_std Rust and Assembly. - Configured two-stage paging with W^X enforcement at Stage-1 and per-VM Stage-2 tables to isolate guest memory. - Implemented a buddy allocator with page coloring to partition the shared L2 cache and reduce jitter for high-priority VMs. - Booted 4 CPU cores using PSCI and pinned GICv2 interrupts to core 0 to prevent jitter on guest cores. - Implemented VM entry/exit paths in Assembly with 928-byte vCPU context switching and ESR_EL2 trap decoding logic. - Enabled direct guest access to PMU counters (cycles, L2 refills) via MDCR_EL2, eliminating EL2 trap overhead.	Mar. 2026 – Current
Bare-Metal x86 Operating System 🌀 <i>C, x86 Assembly, QEMU, Linker Scripts, Makefile</i> - Developed a bare-metal 32-bit monolithic x86 kernel from bootloader to user shell using C and Assembly. - Engineered a two-stage bootloader using Unreal Mode, A20 line switching, and BIOS E820 to enter Protected Mode. - Built a multi-level paging memory manager with bitmap allocation and higher-half mapping for address space isolation. - Implemented Copy-on-Write process forking via reference-counted page directories and page-fault-triggered physical frame duplication, significantly reducing memory consumption and optimizing memory sharing. - Designed a preemptive round-robin scheduler with Ring 0/3 isolation and PIT-driven multitasking for thread execution. - Constructed futex-based synchronization using atomic fast-paths and kernel-level wait queues for high-performance locking. - Programmed drivers for UART, ATA, and PS/2 devices using Port I/O and MMIO, handling interrupts via a custom IDT. - Integrated an ELF binary loader to parse, map, and execute program segments within isolated virtual memory spaces.	Dec. 2025 – Feb. 2026

TECHNICAL SKILLS

Languages: C, x86 Assembly, AArch64 Assembly, Rust, Python, C++, Java
Systems: Kernel Development, Paging, Interrupts, Virtualization, Cache Architecture, Exception Handling
Tools & Environment: Git, objdump, QEMU, Makefile, Linker Scripts, Linux
Embedded & Debugging: Register-Level Debugging, Hardware Interrupt Handling, Cross-Compilation